Zadání 8: Vláknový model

Cíle:
Definovat a implementovat kernel-backed threads

Roztříštit současný model správy procesů na vláknový model

Adam Gabriel & Jan Hejdušek

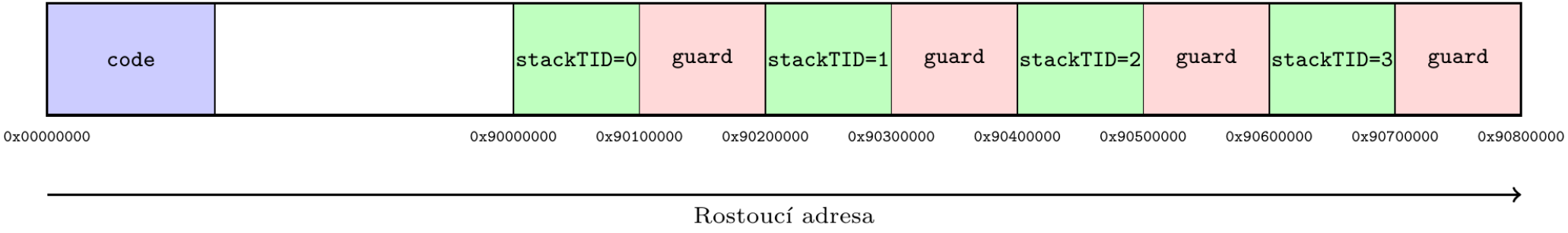

struct TTask_Struct {
TCPU_Context cpu_context;
unsigned int pid;
NTask_State state;
unsigned int sched_counter;
unsigned int sched_static_priority;
IFile* opened_files[Max_Process_Opened_Files
uint32_t multi_wait_fd_bitmap;
bool is_multi_file_wait;
uint32_t multi_wait_ready_fd;
int exit_code;
uint32_t sleep_timer;
uint32_t deadline;
uint32_t notified_deadline;
uint32_t page_count;
struct TProcess_Struct {
uint32_t pid;
uint32_t sched_static_priority;
IFile* opened_files[Max_Process_Opened_Files];
int exit_code;
uint32_t* page_table;
unsigned long ttbr0;
uint32_t page_count;
uint32_t thread_count;
TThread_Struct* threads[Max_Process_Threads];
struct TThread_Struct {
TCPU_Context cpu_context;
TProcess_Struct* process;
unsigned int tid;
NThread_State state;
unsigned int sched_counter;
uint32_t multi_wait_fd_bitmap;
bool is_multi_file_wait;
uint32_t multi_wait_ready_fd;
int exit_code;
uint32_t sleep_timer;
uint32_t deadline;
uint32_t notified_deadline;
unsigned int blocked_on_tid;

Roztříštění Process Mangeru

- nový Process Manager
  - Zakládání, terminace, notifikace všech, soubory (souborový SWI handler)
- Thread Manager
  - Zakládání (první/další), terminace, notifikace, SWI handler (blokující join(), thread(), korektní ukončení vlákna)
- Scheduler
  - Plánování, přepínání, úklid po ukončených vláknech
- + Patřičné úpravy ve zbytku kernelu (zejména sémantika)

Konkrétní problém: Mapování paměti

- Každé vlákno má svůj 1 MB stack, proces má obecnou tabulku stránek
- Ve virt. Paměťovém prostoru „guard intervaly“ – ochrana proti stack overflow (obrázek je blbě)

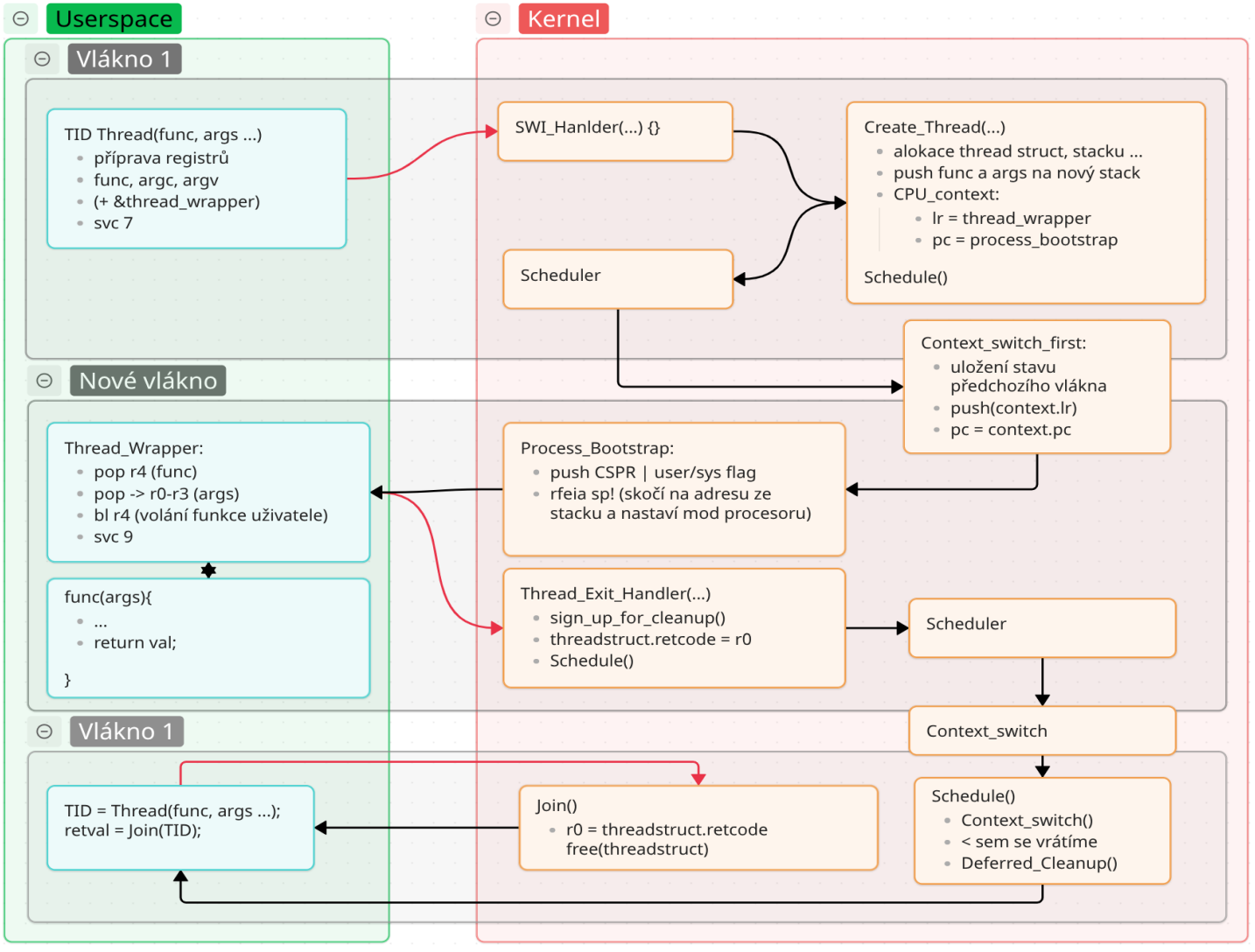


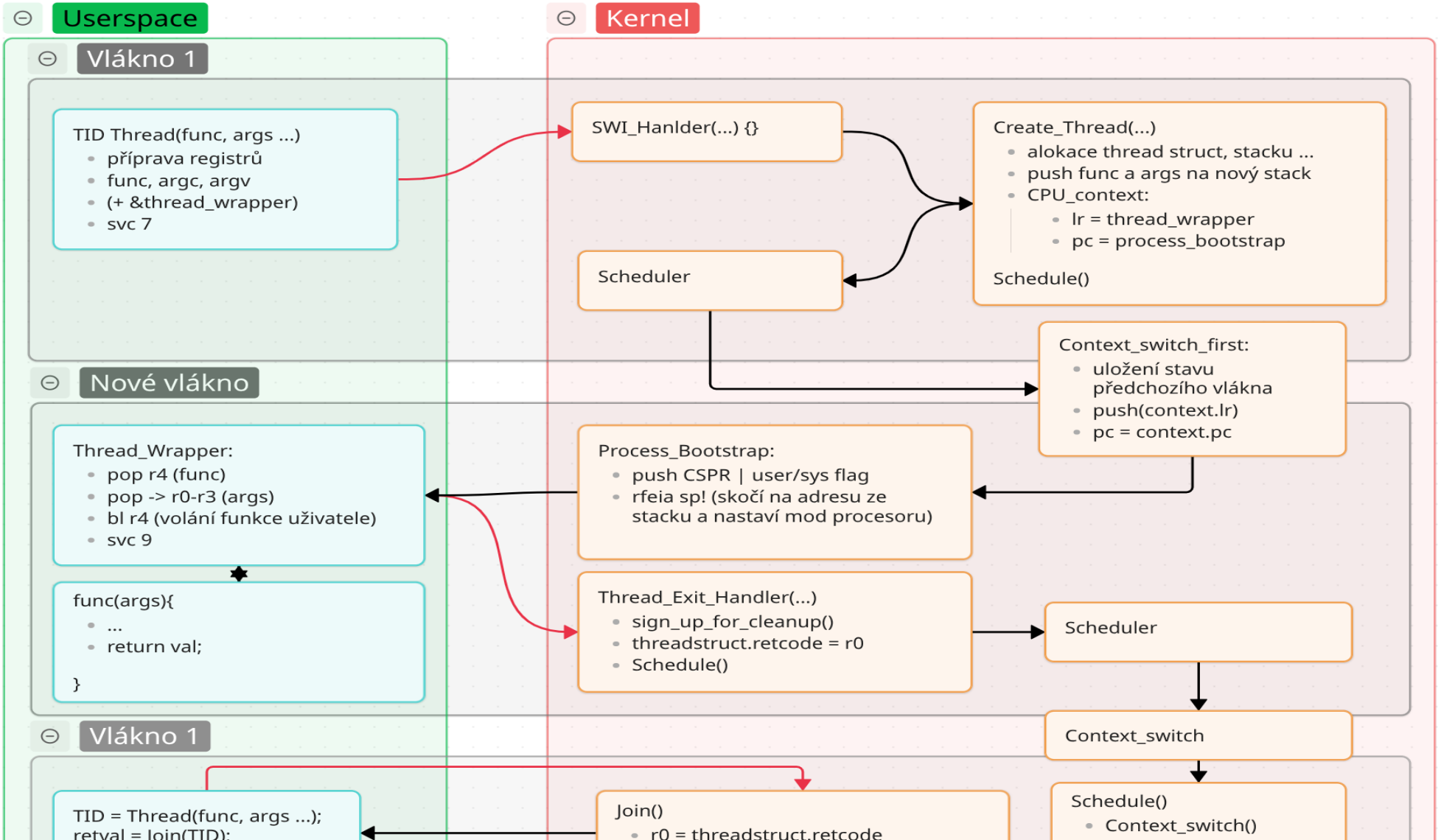
Konkrétní problém: Mapování souborů

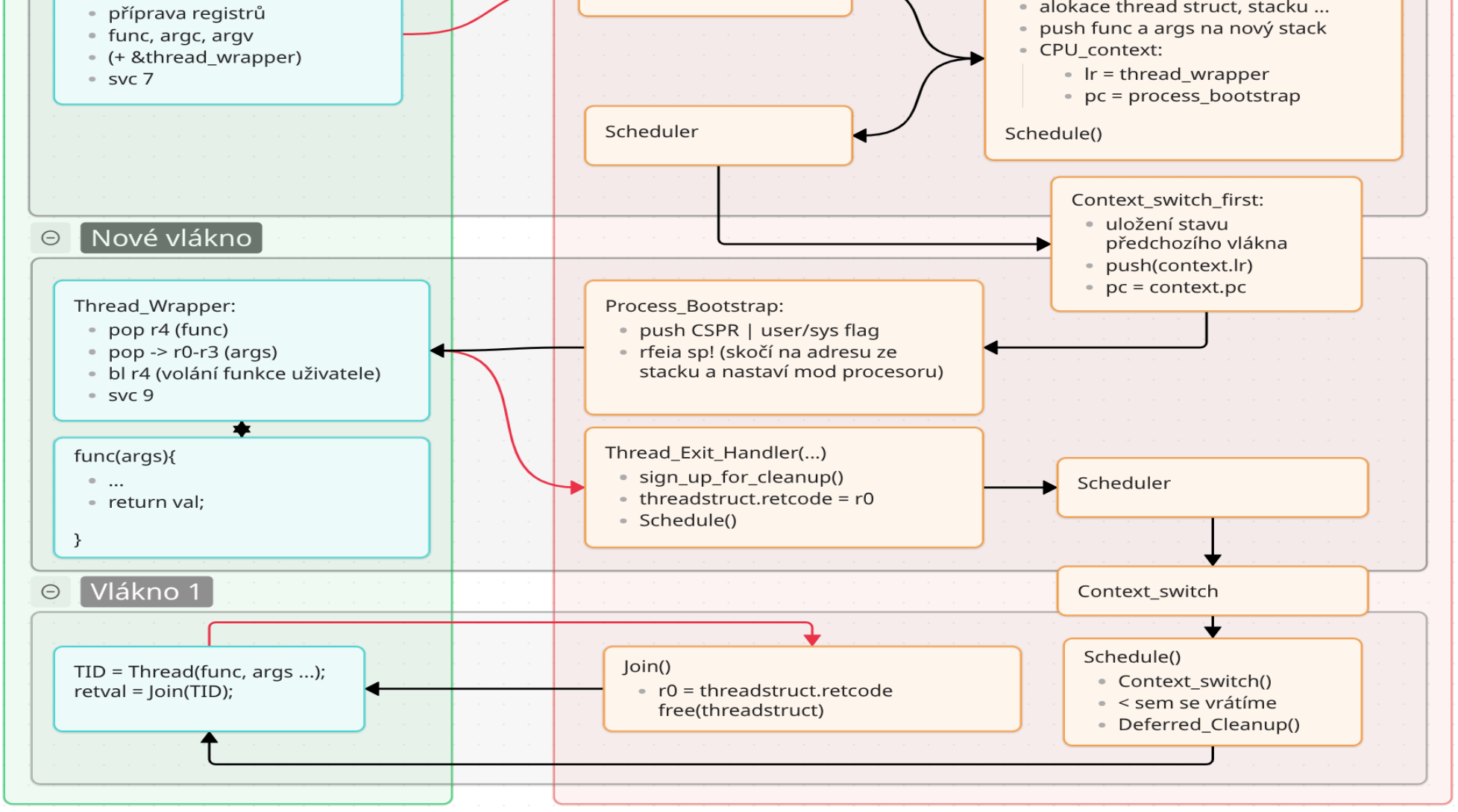
- Více vláken může čekat na jeden soubor
- Jedno vlákno má práva soubor odpamovat
- → Ostatní musím probudit
- → Při mapování dalšího musím kontrolovat, jestli se všechna vlákna probudila

Vytvoření a terminace vlákna

- Z pohledu uživatele jen
  - TID = thread(funkce, argumenty);
  - retval = join(TID);


Userspace
Kernel
Vlákno 1
TID Thread(func, args ...)
příprava registrů
func, argc, argv
(+ &thread_wrapper)
svc 7
SWI_Hanlder(...) {}
Scheduler
Create_Thread(...)
alokace thread struct, stacku ...
push func a args na nový stack
CPU_context:
lr = thread_wrapper
pc = process_bootstrap
Schedule()
Context_switch_first:
uložení stavu předchozího vlákna
push(context.lr)
pc = context.pc
Nové vlákno
Thread_Wrapper:
pop r4 (func)
pop -> r0-r3 (args)
bl r4 (volání funkce uživatele)
svc 9
func(args){
...
return val;
}
Process_Bootstrap:
push CSPR | user/sys flag
rfeia sp! (skočí na adresu ze stacku a nastaví mod procesoru)
Thread_Exit_Handler(...)
sign_up_for_cleanup()
threadstruct.retcode = r0
Schedule()
Scheduler
Context_switch
Vlákno 1
TID = Thread(func, args ...);
retval = Join(TID);
Join()
r0 = threadstruct.retcode
free(threadstruct)
Schedule()
Context_switch()
< sem se vrátíme
Deferred_Cleanup()


Userspace
Kernel
Vlákno 1
TID Thread(func, args ...)
příprava registrů
func, argc, argv
(+ &thread_wrapper)
svc 7
SWI_Hanlder(...) {}
Scheduler
Create_Thread(...)
alokace thread struct, stacku ...
push func a args na nový stack
CPU_context:
lr = thread_wrapper
pc = process_bootstrap
Schedule()
Context_switch_first:
uložení stavu předchozího vlákna
push(context.lr)
pc = context.pc
Nové vlákno
Thread_Wrapper:
pop r4 (func)
pop -> r0-r3 (args)
bl r4 (volání funkce uživatele)
svc 9
func(args){
...
return val;
}
Process_Bootstrap:
push CSPR | user/sys flag
rfeia sp! (skočí na adresu ze stacku a nastaví mod procesoru)
Thread_Exit_Handler(...)
sign_up_for_cleanup()
threadstruct.retcode = r0
Schedule()
Scheduler
Context_switch
Vlákno 1
TID = Thread(func, args ...);
retval = Join(TID);
Join()
r0 = threadstruct.retcode
Schedule()
Context_switch()


příprava registrů
func, argc, argv
(+ &thread_wrapper)
svc 7
alokace thread struct, stacku ...
push func a args na nový stack
CPU_context:
lr = thread_wrapper
pc = process_bootstrap
Schedule()
Scheduler
Context_switch_first:
uložení stavu předchozího vlákna
push(context.lr)
pc = context.pc
Nové vlákno
Thread_Wrapper:
pop r4 (func)
pop -> r0-r3 (args)
bl r4 (volání funkce uživatele)
svc 9
Process_Bootstrap:
push CSPR | user/sys flag
rfeia sp! (skočí na adresu ze stacku a nastaví mod procesoru)
func(args){
...
return val;
}
Thread_Exit_Handler(...)
sign_up_for_cleanup()
threadstruct.retcode = r0
Schedule()
Scheduler
Context_switch
Vlákno 1
TID = Thread(func, args ...);
retval = Join(TID);
Join()
r0 = threadstruct.retcode
free(threadstruct)
Schedule()
Context_switch()
< sem se vrátíme
Deferred_Cleanup()